

Mini Project (HPC)

```
In [1]: # Name: Atharv Santosh Danave
# Roll No.: 11
# Project Title: Evaluate performance enhancement of parallel Quicksort Algorithm using MPI
```

```
In [3]: # Install MPI (run once)
!apt-get update
!apt-get install -y openmpi-bin libopenmpi-dev
```

0% [Working]

```
Hit:1 https://cloud.r-project.org/bin/linux/ubuntu jammy-cran40/ InRelease
Get:2 https://cli.github.com/packages stable InRelease [3,917 B]
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:5 https://r2u.stat.illinois.edu/ubuntu jammy InRelease
Hit:6 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:7 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu jammy InRelease
Hit:8 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntugis/ppa/ubuntu jammy InRelease
Fetched 3,917 B in 1s (3,297 B/s)
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu jammy InRelease' does not seem to provide it (sources.list entry misspelt?)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libopenmpi-dev is already the newest version (4.1.2-2ubuntu1).
libopenmpi-dev set to manually installed.
openmpi-bin is already the newest version (4.1.2-2ubuntu1).
openmpi-bin set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 34 not upgraded.
```

```
In [4]: # Write the MPI C program to a file

code = r'''
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Comparator for qsort
int compare_ints(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

// Merge two sorted arrays
int* merge(int *a, int n, int *b, int m) {
    int *result = malloc((n + m) * sizeof(int));
    int i = 0, j = 0, k = 0;

    while (i < n && j < m)
        result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];

    while (i < n) result[k++] = a[i++];
}
```

```

        while (j < m) result[k++] = b[j++];

        return result;
    }

int main(int argc, char *argv[]) {
    int rank, size;
    int N = 100000; // Total elements

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int local_n = N / size;

    int *data = NULL;
    int *local_data = malloc(local_n * sizeof(int));

    double start, end;

    // Root generates data
    if (rank == 0) {
        data = malloc(N * sizeof(int));
        for (int i = 0; i < N; i++)
            data[i] = rand() % 1000000;

        start = MPI_Wtime();
    }

    // Scatter data
    MPI_Scatter(data, local_n, MPI_INT,
                local_data, local_n, MPI_INT,
                0, MPI_COMM_WORLD);

    // Local sorting using quicksort
    qsort(local_data, local_n, sizeof(int), compare_ints);

    int *gathered = NULL;
    if (rank == 0)
        gathered = malloc(N * sizeof(int));

    // Gather sorted chunks
    MPI_Gather(local_data, local_n, MPI_INT,
                gathered, local_n, MPI_INT,
                0, MPI_COMM_WORLD);

    // Merge all sorted parts
    if (rank == 0) {
        int *sorted = malloc(local_n * sizeof(int));
        memcpy(sorted, gathered, local_n * sizeof(int));

        for (int i = 1; i < size; i++) {
            int *merged = merge(sorted, i * local_n,
                                gathered + i * local_n,
                                local_n);

            free(sorted);
            sorted = merged;
        }
    }
}

```

```

        end = MPI_Wtime();

        printf("Processes: %d | Time: %f sec\n", size, end - start);

        free(sorted);
        free(data);
        free(gathered);
    }

    free(local_data);
    MPI_Finalize();
    return 0;
}
...

with open("mpi_qsort.c", "w") as f:
    f.write(code)

```

```

In [5]: # Compile the MPI program
!mpicc mpi_qsort.c -o mpi_qsort

```

```

In [6]: # Run with 1 process
!mpirun --allow-run-as-root --oversubscribe -np 1 ./mpi_qsort

# Run with 2 processes
!mpirun --allow-run-as-root --oversubscribe -np 2 ./mpi_qsort

# Run with 4 processes
!mpirun --allow-run-as-root --oversubscribe -np 4 ./mpi_qsort

```

```

Processes: 1 | Time: 0.014146 sec
Processes: 2 | Time: 0.009097 sec
Processes: 4 | Time: 0.013484 sec

```

```

In [7]: # Run multiple times for better comparison
for p in [1, 2, 4, 8]:
    print(f"\nRunning with {p} processes")
    !mpirun --allow-run-as-root --oversubscribe -np $p ./mpi_qsort

```

```

Running with 1 processes
Processes: 1 | Time: 0.048302 sec

```

```

Running with 2 processes
Processes: 2 | Time: 0.009506 sec

```

```

Running with 4 processes
Processes: 4 | Time: 0.016069 sec

```

```

Running with 8 processes
Processes: 8 | Time: 0.013926 sec

```

```

In [ ]:

```